



Original software publication

A new approach for drone tracking with drone using Proximal Policy Optimization based distributed deep reinforcement learning

Ziya Tan^{a,*}, Mehmet Karaköse^b^a Tokat Gaziosmanpaşa University Tokat, Turkey^b Firat University Elazığ, Turkey

ARTICLE INFO

Article history:

Received 15 March 2023

Received in revised form 29 July 2023

Accepted 6 August 2023

Keywords:

Distributed learning

Drone tracking

Reinforcement learning

Proximal Policy Optimization

ABSTRACT

In this paper, a distributed deep reinforcement learning algorithm based on Proximal Policy Optimization (PPO) is proposed for an unmanned aerial vehicle (UAV) to autonomously track another UAV. Accordingly, this paper makes three important contributions to the literature. The first one is the development of an efficient UAV tracking algorithm, the second one is the presentation of a deep reinforcement learning approach that can be adapted to the problem, and the third one is the introduction of a generalized distributed deep reinforcement learning platform that can be used in various problems such as tracking, control and mission coordination of UAVs. In order to validate the developed approaches, the PPO algorithm is simulated with the deep reinforcement learning algorithm in a distributed and non-distributed manner, a follower UAV is trained in different scenarios and the distributed and non-distributed performances of the training using CPU are obtained, scenarios using general and adaptive learning algorithms are given, and finally, the performances of the algorithms developed in the paper are presented explicitly.

© 2023 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Code metadata

Current code version

Permanent link to code/repository used for this code version

Permanent link to reproducible capsule

Legal code license

Code versioning system used

Software code languages, tools and services used

Compilation requirements, operating environments and dependencies

If available, link to developer documentation/manual

Support email for questions

V1.0

<https://github.com/ElsevierSoftwareX/SOFTX-D-23-00170>https://github.com/ziya44/take_off_drone.git

MIT Licence

None

Python

Ubuntu 20.04

ziya.tan@gop.edu.tr

1. Motivation and significance

Developed in the early 1980s, Reinforcement Learning (RL) [1] is one of the important areas of machine learning. RL is a subfield of Machine Learning in which an agent interacting with an environment learns the optimal action selection policy by receiving feedback from the environment. RL algorithms learn iteratively: at each iteration, the agent observes its state and chooses an action that leads to the state of the subsequent environment. Then,

the agent receives a reward or penalty and updates the value function representing the goal to maximize. By exploring the environment and gathering this information, agents learn how to maximize long-term reward. Fig. 1 shows the agent-environment interaction for reinforcement learning approaches.

The benefits of distributed systems in reinforcement learning are demonstrated through the RLlib [2] library. The use of distributed systems in solving such complex problems will enable more parameters to be trained in less time. Therefore, learning the environment will be easier. Especially for UAVs, the importance of distributed systems comes to the forefront when it is considered that more than one algorithm needs to be trained at the same time. This approach decentralizes the learning event, in

* Corresponding author.

E-mail addresses: ziya.tan@gop.edu.tr (Ziya Tan), mkarakose@firat.edu.tr (Mehmet Karaköse).

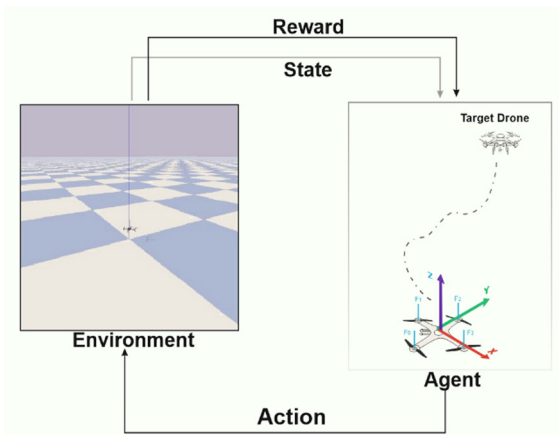


Fig. 1. Agent-environment interaction for reinforcement learning.

other words, it solves the same problem by using more workers instead of one worker. Thus, the learning event is optimized and information sharing is done faster.

In [3], an in-depth literature survey was conducted to examine the techniques, datasets and hardware used to track multiple objects on busy roads. In the paper [4], where the problem of urban public transportation is considered, a Markov decision process (MDP) is used to solve the problem. In addition, a distributed reinforcement learning approach was used to overcome the high computational cost. In [5], where distributed deep reinforcement learning is used to optimize wireless network sensors, a wireless medium access control layer is learned to maximize the throughput of network nodes. Akloufi et al. [6] used deep reinforcement learning algorithms to optimize the actions of another drone during tracking, and as a second approach, they developed a search space proposal to determine the location of the drone to be tracked. In [7], a proposal for drone tracking using a millimeter radar system is presented and preliminary results of tracking a drone using a radar mounted on another drone are presented. In [8], a distributed reinforcement learning approach is presented to address the collaborative sensing and transmission problem for drones. A distributed sense and transmit protocol is proposed to coordinate multiple UAVs. A distributed deep reinforcement learning approach for autonomous aerial vehicles is presented in [9], where the modeled approach is proposed to optimize the take-off and passenger pick-up of an air taxi to pick up its customers from the optimal location. In the study [10], which proposes a parameter sharing and behavior-based method for swarm robots, the PPO algorithm is tested in different environments and the results are observed. In the study [11], a new deep reinforcement learning architecture for the control and optimization of UAV networks is presented and it is observed that the use of distributed architecture improves the performance up to 3.7 times. In Venturini's master thesis [12], a distributed deep reinforcement learning approach was used for swarm drones. In this thesis, all the typical challenges of a reinforcement learning framework were met. The work by Seejong Park et al. in [13] presents a comprehensive survey on systems against unmanned aerial vehicles. The simulator environment used includes many obstacles such as trees, cables, parked cars and houses. In [14], an analysis of recent DRL studies, the policies used are analyzed. In the study, the policies used are divided into three groups and the problem of moving agents to a common goal is discussed using Newton's law of motion. In the study [15], which trained three different robots with DRL method in limited areas using the policy gradient algorithm, the performance of the robots was evaluated and successful results were obtained. An optimized version of the

distributed PPO algorithm is used in the article [16], which is prepared to help an agent learn complex behaviors in an environment with different barriers. Trained by the distributed PPO algorithm tested in various obstacles and terrains, the agent improved movement skills thanks to simple reward functions. The agent has successfully learned tasks such as jumping, running, and crouching without any direction.

Table 1 compares the methods and results of some similar studies. Although different methods are used in drone tracking, it is seen that YOLO deep learning algorithm is generally used in real-time drone detection algorithms.

In this study, we compare the performance of DDRL and non-distributed deep Reinforcement learning algorithms to successfully overcome one of the biggest problems faced by quadcopters, namely, take-off and tracking. In the experiments with CPU, it is observed that the use of distributed architecture improves the training performance and reduces the training time. The main contributions of this paper can be summarized as follows.

- To present the use of DRL in drone tracking and vertical take-off problems and the performance of distributed systems,
- Comparison of PPO-based distributed and non-distributed approaches
- Being a source literature for similar problems to be done in the future

2. Software description

In this section, algorithms and basic concepts of the software used in the article are explained. First of all, information about RL is given. Secondly, detailed information about deep reinforcement learning is given. Finally, the DDRL approach used in this study is examined. In this study, mainly python software language, torch framework and gym-pybullet simulation environment are used. In addition, the ray framework was used to distribute the existing architecture.

2.1. Software architecture

This study basically examines UAV navigation in 3D environment. This navigation process was basically compared using two different approaches. Distributed deep reinforcement learning and deep reinforcement learning approaches train an agent for one drone to follow another. As a result of the training, the follower drone aims to successfully follow at a certain altitude by positioning itself according to the followed drone. For this purpose, reinforcement learning and PPO algorithm are used together. RAY framework was preferred as distributed architecture. The workers in the training environment perform the action evaluated by the deep neural network. The reward and status values received from the environment as a result of the action are evaluated by the PPO architecture. According to the rewards and punishments received as a result of these actions, the agent learns the environment and acts accordingly. In this study, training takes place with a maximum of 4 workers, depending on the processor. This number can increase even more when GPU is used. These workers learn the environment independently of each other and transmit the results to a common learner. Thus, a faster and more successful training is provided in a unit of time. In the non-distributed model, a single agent performs the actions in the environment.

Fig. 2 shows the basic software architecture of drone tracking used in our study. Within the scope of the study, the training of which is carried out in the virtual environment, the follower drone performs its action according to the followed drone. The

Table 1
Comparison of similar DDRL studies.

Paper	Issue	Method	Result
[6]	UAV pursuit-evasion	Deep Reinforcement Learning and Search Area Proposal	Proposed approach is effective in tracking moving objects in complex outdoor scenarios
[7]	To detect and follow target drones	Experimentally validate an airborne drone detection system that utilizes a millimeter wave radar system	Was not able to detect the drone at long distances
[17]	Autonomous drone detection and tracking system	YOLO deep learning algorithm is used for drone detection and newly developed target candidate tracking (TCT) algorithm is used for drone tracking.	This system, in collaboration of a static wide-angle camera and a rotating low-angle camera, has been proven to provide plausible results based on simulations and field tests.
[18]	Autonomously detects, hunts, and takes down other small UAVs in GPS-denied environments	Was used to train a Tiny YOLO detection algorithm.	The detection accuracy of the present system is 77%. The detection speed, 8.33 fps,
[8]	Are considered a cellular Internet of UAVs, where the UAVs execute sensing tasks through cooperative sensing and transmission to minimize the age of information (Aol).	Are formulated the trajectory design problem for Aol minimization and proposed a compound action distributed actor-critic (CA2C) algorithm to solve it based on deep reinforcement learning.	Are shown that by dividing the tasks, cooperative UAVs can achieve a lower Aol compared to non-cooperative UAVs.
[19]	Setting and enabling distribution for training to reduce the time of the experiment.	Comparison of distributed and non-distributed reinforcement learning algorithm.	Distributed architecture is more successful than non-distributed architecture.
[20]	Traffic light control	Using distributed deep reinforcement learning method	The CNN-based Q network is optimized due to the interaction of the agent trained for the traffic light with the environment. In the simulation where the proposed distributed architecture is compared with the centralized learning algorithms, it has been concluded that the distributed learning algorithm performs better.
[21]	A 4-stage method is presented as a solution to the Hybrid Flow Shop (HFS) scheduling problem under real-world conditions.	The concepts presented in the paper are developed by testing a DRL agent in a game-like environment. PPO and A3C algorithms are used to solve the complexity of the problem.	As a result of the experiments, it was observed that the agents trained by both algorithms experienced performance degradation in solving problems of different natures. Still, the adjusted policies improved the performance after a certain period. As a result, it is concluded that more research is needed to improve the performance of DRL agents.
[22]	Solving the problem by training an RL agent with one set of parameters in multitasking definitions.	IMPALA, a new distributed architecture with high scalability, is introduced. We also present V-trace, an out-of-policy learning algorithm.	The experiments show that the introduced distributed algorithm (IMPALA) performs better than the others. It also exhibits positive transfer between distributed tasks.
[23]	Developing traffic signal control to increase efficiency to solve the transportation problem in megacities.	An RL model is proposed using Ape-X and Deep Q-learning networks on small data sets.	As a result of the experiments conducted on datasets from three different cities, it was found to be more successful than existing RL-based methods. In addition, the average travel times in these cities have also reached a better point.
[24]	To compare UAV navigation frameworks and simulation software available in the literature.	Comparison of UAV navigation and simulators and performance measurements of RL algorithms.	As a result of the analysis, some challenges and advantages were identified. In particular, it has been observed that the complex tasks of UAVs performed by RL algorithms cause performance degradation due to the inadequacy of the simulation environment.

artificial neural network, which is the action determinant, determines action in line with the determined parameters and enables the drone to move in the environment. The reward he will receive from this action will be decisive in his next actions.

This article uses the Gym toolkit developed by OpenAI. It is used mainly in RL problems and in developing existing solutions using Python. PyBullet is used to model the forces and torque acting on the drone created in Gym. "OpenAI's" Gym toolkit

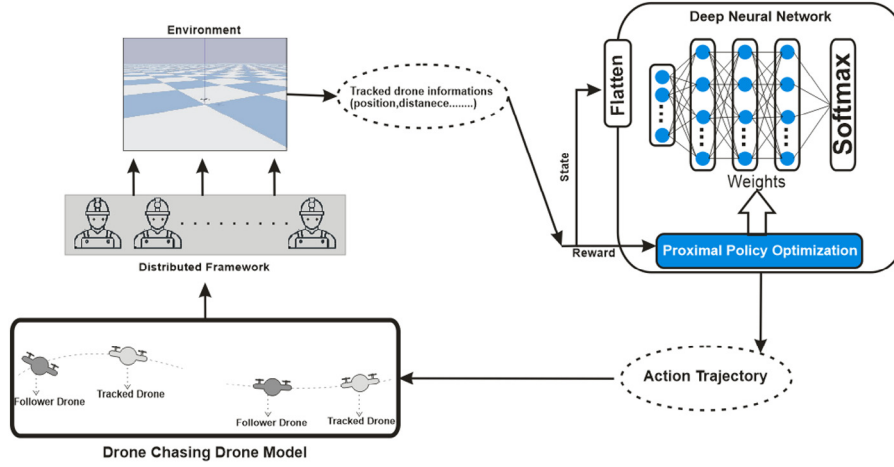


Fig. 2. Architecture of drone tracking drone model.

was introduced to standardize the development of RL problems and algorithms in Python. We use PyBullet to model the forces and torques acting on each quadcopter in our Gym and leverage the physics engine to calculate and update the kinematics of all the vehicles. During the agent's training, as in the standard operation in an MPD framework, an action is taken from the environment, which updates the internal state. The result returns a reward and state corresponding to the action taken. This repeats throughout the set cycle. The reward value returned at each step is determined by the function described in Section 2.2.3. The upward movement of the leader drone will teach the follower drone, which is in the starting position, the path to follow.

2.2. Software functionalities

In this section, the functions of the software used in the study carried out for drone tracking are explained

2.2.1. Distributed deep reinforcement learning

In RL theory, unlike other machine learning methods, no prior knowledge is given. The aim is to maximize the rewards for future actions by using the experiences gained after exploring the environment. There are two basic elements in RL, the agent and the environment. Two distinctive features of RL; trial and error search and delayed reward feature [25].

Agent: A learner and decision maker in an environment. Sometimes the agent can be a drone performing a task or a robot learning to stand.

Environment: The world created for an agent to learn something. This can sometimes be the exact environment observed by the agent, such as in arcade games. Sometimes it can be a partially observed environment, such as a camera image.

The agent and the environment interact at each t time period. At each step t , the agent receives a representation of the environment $s_t \in S$, and chooses an action from the set $A(S)$. One step later, the agent receives a numerical reward ($r_t \in R$) as a result of the interaction and sets a new state (s_{t+1}) [26]. In Fig. 3, the operation of the DRL is visualized.

Deep Reinforcement Learning (DRL) is basically the same as RL. However, the main difference is that it uses deep learning algorithms instead of experience tables for action choices. The best example of this is the Atari games developed by Google DeepMind. Especially in the fields of computer vision, voice recognition, and robotics, where great strides have been made in recent

years, large datasets need to be trained. Normally the estimation of the value function can be represented in a tabular form.

However, the large continuous state spaces that occur in real-world problems slow down the training considerably. In this case, the problem is solved using neural networks [27]. The aim of the agent is to make the neural network closer to the optimal action-value function by maximizing the cumulative reward, as shown in Eq. (1).

$$Q'(s, a) = \max_{\pi} \mathbb{E} \{ r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s, a_t = a, \pi \} \quad (1)$$

With the developments in machine learning, especially in the field of robotics, DRL had to improve itself. Distributed RL has succeeded in increasing the pace of learning by distributing learning among many workers. In addition, environmental knowledge learned per unit time also increases. Gorilla is one of the pioneering architectures in this field. After Gorilla, current distributed architectures such as A3C, IMPALA, APE-X, D4PG have been used. In this architecture, the goal is to gain experience using multiple actors to update the weights [8]. All used actors can access a trained system at the same time. Distributed Deep Reinforcement learning architecture is described in algorithm 1 and algorithm 2 [28].

Algorithm 1 Worker k : ComputeGradient
state: Replay dataset D_k , game state s_t , target model θ , target model generation ℓ
Fetch model θ' and iteration number n from server.

If $n \geq \ell C$ **then**
 $\theta' \leftarrow \theta$
 $\ell \leftarrow [n/C] + 1$
 $q \leftarrow \max_{N_{\max}} \{ \frac{(N_{\max} - n)}{N_{\max}}, 0 \}$
 $\varepsilon \leftarrow (1 - q) \varepsilon_{\min} + q \varepsilon_{\max}$
Select action $a \leftarrow \begin{cases} \max_a Q(\theta(s_t), a; \theta) & w.p. 1 - \varepsilon \\ \text{random action} & \text{otherwise} \end{cases}$

Execute action a and observe reward r_t and frame x_{t+1}
Append $s_{t+1} = (s_t, a_t, x_{t+1})$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
Store experience $(\phi_t, a_t, r_t, \phi_{t+1})$ in D_{k-1}
Uniformly sample minibatch of experience X from D_k , where $X_j = (\phi_j, a_j, r_j, \phi_{j+1})$
Set $y_j = \begin{cases} r_j & \text{if } \phi_{j+1} \text{ terminal} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta') & \text{otherwise} \end{cases}$

$$\Delta \theta \leftarrow \nabla_{\theta} f(\theta; X) = \frac{1}{b} \sum_{i=1}^b \nabla_{\theta} \left(\frac{1}{2} (Q(\theta_i, a_i, \theta) - y_i)^2 \right)$$

Send $\Delta \theta$ to parameter server

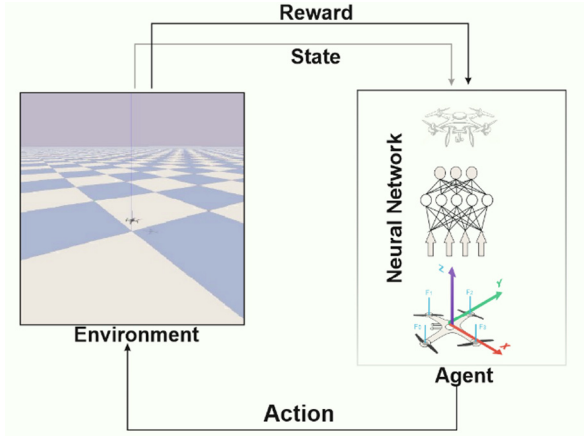


Fig. 3. Reward-action loop for DRL.

Algorithm 2 Distributed deep reinforcement learning

```

Function update( $\Delta\theta$ )
 $r_i \leftarrow 0.9r_i + 0.1(\Delta\theta)_i^2$  for all  $i$ 
Acquire write-lock
 $\theta_i \leftarrow \theta_i - \alpha(\Delta\theta)_i / \sqrt{r_i}$ 
Release write-lock
Initialize server  $\theta_i \sim \mathcal{N}(0, \zeta), r \leftarrow 0, n \leftarrow 0$ 
for all workers  $k$  do
  AsyncStart
  Populate  $D_k$  by playing with random actions
  repeat
    ComputeGradient
  until server timeout
  AsyncEnd
While  $n < \text{MaxIters}$  do
  if latest model requested by worker  $k$  then
    Acquire write-lock
    Send  $(\theta, n)$  to worker  $k$ 
    Release write-lock
  if gradient  $\Delta\theta$  received from worker  $k$  then
    Update( $\Delta\theta$ )
     $n \leftarrow n+1$ 
Shutdown server

```

2.2.2. Proximal policy optimization (PPO)

PPO is a gradient-based reinforcement learning method. This method aims to optimize the objective function using stochastic gradient descent. During the training phase, the PPO agent estimates the probabilities for each action that might take place and randomly selects the action based on this value. Similar to the Trust Region Policy Optimization (TRPO) objective function, Eq. (2) defines the odds ratio between the new policy and the old policy.

$$r(\theta) = \frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} \quad (2)$$

The objective function of TRPO can be updated as in Eq. (3).

$$j(\theta)^{TRPO} = E[r(\theta) \hat{A}_{\theta_{old}}(s, a)] \quad (3)$$

Without some constraints, this objective function can cause instability or slow convergence rate. The PPO uses the policy rate $(r(\theta))$, which is about 1 without complex restrictions. This range is set from $1-\varepsilon$ to $1+\varepsilon$ thanks to the ε hyper parameter, which is set to 0.2 by default. After these adjustments, the objective function of PPO can be written as in Eq. (4).

$$J^{CLIP}(\theta) = \mathbb{E}[\min(r(\theta) \hat{A}_{\theta_{old}}(s, a), \text{clip}(r(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_{\theta_{old}}(s, a))] \quad (4)$$

Table 2

Architecture for PPO algorithm.

Parameter	Value
Hidden layer	256
Activation function	tanh
Central critic	25 inputs
Action models	12 inputs

In Eq. (4), the clip function determines the policy ratio with values between $[1-\varepsilon, 1+\varepsilon]$. Since the PPO objective function takes the minimum value between the original and clipped values, it prevents bias toward a negative action.

2.2.3. Reward function

During the tracking process, only the x y z position and yaw angle of the drone approaching the target are adjusted. The flight controller automatically adjusts the roll and pitch angle to follow the target, moving the fighter to the next set point. Unless the drone moves forward, the roll and pitch remain zero and the drone maintains its position. In the LeaderFollower environment, the reward function consists of the rewards earned by the follower drone by following the altitude of the tracked drone. The reward function is shown in Eq. (5).

$$r_0 = -\|[0, 0, 0.6] - x_0\| \frac{2}{\sqrt{2}}, \quad r_1 = -0.6(z_1 - z_0)^2 \quad (5)$$

2.2.4. Hyperparameters of the software

During the training, the architecture of the PPO algorithm in both experiments was arranged as shown in Table 2. The training was conducted on a computer with an Intel Core i7-5500 CPU 2.4GH*4 processor.

The first experiment is created to measure the effect of reinforcement learning used in drone simulation. Thus, an advanced experiment is presented on quadcopter UAVs using reinforcement learning algorithms to achieve the goal and complete the mission. In this experiment, the training was designed non-distributed and used only the CPU. The parameters used for the DDRL algorithm trained as 300000 steps are shown in Table 3.

The second experiment distributed the PPO algorithm among more than one worker. The only difference in the parameters used according to the first experiment is the number of workers. In the second experiment, the number of workers was determined as 4. Thus, we can compare the results of the two experiments.

In this experiment, the GPU was not used. The agent is trained by four different workers with the help of distributed architecture using only CPU. The experiment was trained for 300000 time steps and the rewards the agent received from the environment were given compared with the first experiment data.

3. Illustrative examples

The software used in this study addresses motion control problems, object tracking, and vertical ascent for a quadcopter. A gradient-based deep RL approach is used to generate a control strategy. This paper presents two different approaches to train the DRL algorithm. The first approach trains the learning algorithm together with a distributed architecture. The second one is trained in a non-distributed manner. The training is performed using CPU. In order to increase the success of training and reduce the computational cost in such complex problems, we have shown through experiments that the CPU alone cannot be successful and can be more successful with multiple workers when

Table 3
Used parameters for DDRL.

Parameter	Value	Explanation
env	LeaderFollower	
Env_type	OpenAI Environment	
Executed algorithm	PPO	
Stop condition	Time-steps = 300000	
lambda	1.0	The GAE (generalized advantage estimation) parameter
train_batch_size	400	Size of each SGD (stochastic gradient descent) epoch
learning rate	5e−5	Learning rate for training
kl_coeff	0.2	Initial koefficient for KL divergence
Num_sgd_iter	30	Number of sgd iterations in each outer loop
Num_gpu	0	
worker	0	

Table 4
Comparison of existing papers on the study of UAV drone tracking.

Paper	Algorithm	Performance values
[29]	CNN-LSTM (distributed)	The accuracy rate is ~%98
[30]	DDPG	35% of the successful tracking ratio
[31]	SAC	The average reward is 32
[32]	UCT (distributed)	75% of the successful tracking ratio
[33]	DRL	The cumulative total reward is 73.70
[34]	CMDP	28% searching time reduction
[35]	MDP	Control accuracy (m/s ²) is 0.2691
Our study	Distributed PPO	The average reward is −54.78

trained in a distributed manner. We can follow the experiments in three different axes. Accordingly, the follower drone performed the tracking function in the vertical plane for a period of 6 s. During this time, the second experiment was about ten times more successful than the first.

Table 4 shows the results of some special studies in the literature similar to the experiments conducted in this paper according to the environment and DRL algorithms used. When Table 4 is analyzed, it is seen that different effects, such as average reward or task success rate, are obtained according to the environment in which reinforcement learning is used. The fact that distributed systems have not been used much in the drone tracking problem makes it difficult to compare the results. In addition, since the approach proposed in our paper is to prove the success of the distributed system over the non-distributed system in drone tracking, it is impossible to compare it with similar problems.

4. Impact

This study addresses problems such as motion control, object tracking, and vertical elevation for a quadcopter. A gradient-based deep RL approach is used to establish a control strategy. This article presents two different approaches to training the DRL algorithm. The first approach trains the learning algorithm with a distributed architecture. The second is trained as non-distributed. The training was carried out using the CPU. To increase the success of the training and reduce the computational cost in such complex problems, we have shown with experiments that the CPU cannot be successful alone and can be more successful with more than one worker when trained in a distributed manner. We can follow the experiments on three different axes. Accordingly,

the follower drone performed the tracking function in the vertical plane in a time frame of 6 s. In Fig. 4, drone tracking is simulated.

The software is a piece of code that can be easily used and installed by the user without requiring anything other than some basic requirements. In particular, they can customize the artificial neural network and change the simulation environment. Therefore, it can be widely used in academic and scientific studies. The software also makes it easier for researchers to combine distributed architecture and reinforcement learning algorithms. Since the algorithm trained using Python code is open source, other users can easily update and improve it. This software is shipped fully open source using the MIT license and an explanatory user manual with links to all the libraries used.

4.1. Experimental results

In both experiments, while 300000 time-step training was performed, the division-based average of the rewards received by the agent from the environment is shown in Fig. 5. While the agent was trained for a total of 700 episodes during the training phase in the first experiment, this value was approximately 500 episodes in the second experiment. The distributed-trained agent completes more Episodes in the same time slot. When Fig. 5 is examined, it is concluded that the mean reward value of the distributed PPO algorithm during the training is −54.78 and that the non-distributed PPO algorithm is −559.49.

Comparing the results in this experiment, training time, rewards received in episodes, and solving the given task, It was found that the second experiment gave more successful results than the first. Training for the second experiment took approximately 1 h.

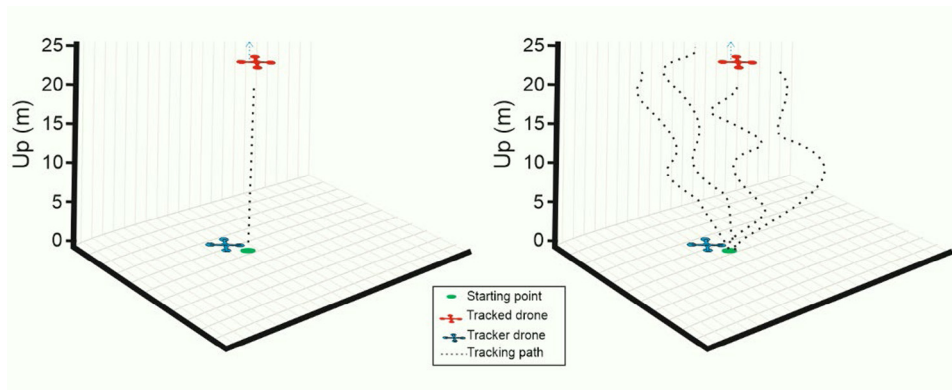


Fig. 4. Simulations demonstration for drone tracking.

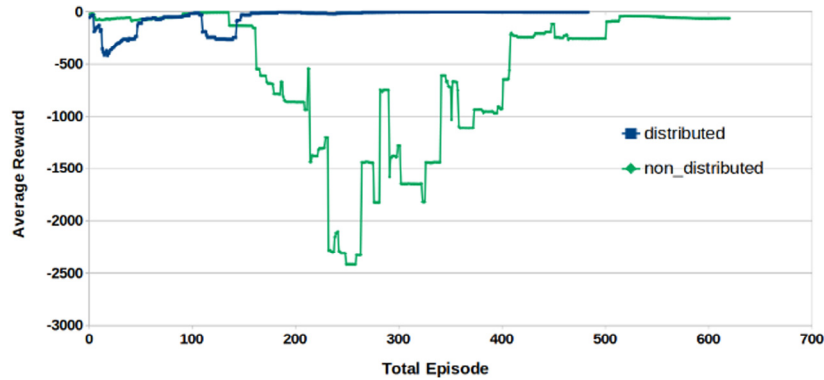


Fig. 5. Average reward for each comparative episode received in both experiments.

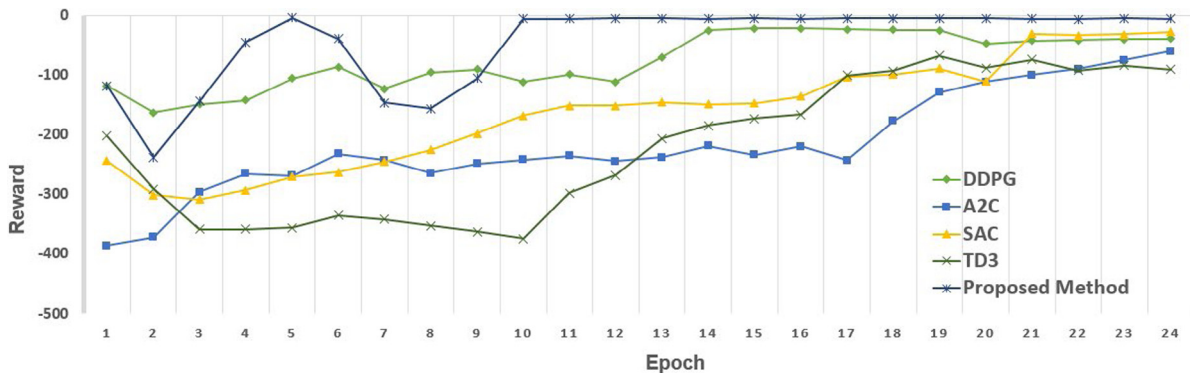


Fig. 6. 20-step reward averages for the use of five DRL algorithms in a stable-baselines3 framework.

We also compare the proposed method with A2C [8], DDPG [30], SAC [31], and TD3 [36] using the stable-baselines3 [37] framework, an open-source implementation of the most advanced deep reinforcement learning algorithms. The experiments are trained for 500 steps with the same hyper-parameters. The results are shown in Fig. 6.

The average rewards the agent receives during training are given in Table 5.

When Fig. 6 and Table 5 are analyzed, it is seen that the proposed method maximizes the reward value of the agent earlier than other state-of-the-art drl algorithms, and the average reward value is higher during training.

5. Conclusions

This paper develops an application that supports distributed architectures, enabling researchers to get more out of CPU-based

Table 5

Average reward values in the training phase of the algorithms using the stable-baselines3 framework.

Algorithm	Mean reward
SAC	-163,24
TD3	-221,54
DDPG	-75,62
A2C	-216,36
Proposed method	-54,78

deep reinforcement learning software. In this context, the problem of drone tracking with a drone is addressed and two different experiments are conducted it is observed that the distributed

deep reinforcement learning software is more successful than the non-distributed software. According to the data obtained from the experiments, the number of steps performed in each iteration is approximately 67% higher in distributed training. According to the reward graphs of the training results, the distributed algorithm was more successful. In the algorithms trained in 300000-time steps, the second successful experiment had an average of -54.78 rewards during training. This value is -559.49 for the non-distributed algorithm.

According to the test results in Section 4.1, it was observed that the success of the first experiment in the training phase did not change in the test results. Accordingly, it is concluded that the UAV trained with the distributed PPO algorithm is more successful in tracking the target drone. Our study will contribute to filling the gap in the literature on the synchronized use of reinforcement learning with distributed systems in unmanned aerial vehicles. It will also help in developing realistic DDRL applications for unmanned aerial vehicles.

Our future plan is to develop a novel algorithm for drone detection and tracking so that the tracking drone can perform its mission successfully regardless of the environment.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This study was inspired by the Ph.D. thesis study of Ziya TAN titled “Distributed Deep Reinforcement Learning Approaches for Drone Tracking”.

References

- [1] Cao Z, Guo H, Song W, Gao K, Chen Z, Zhang L, et al. Using reinforcement learning to minimize the probability of delay occurrence in transportation. *IEEE Trans Veh Technol* 2020;69(3):2424–36.
- [2] Liang E, Liaw R, Moritz P, Nishihara R, Fox R, Goldberg K, et al. RLlib: Abstractions for distributed reinforcement learning. 2018, arXiv:1712.09381.
- [3] Jiménez-Bravo DM, Murcigo ÁL, Mendes AS, Blás HS, Bajo J. Multi-object tracking in traffic environments: A systematic literature review. *Neurocomputing* 2022;494:43–55.
- [4] Yan H, Cui Z, Chen X, Ma X. Distributed multi-agent deep reinforcement learning for multi-line dynamic bus timetable optimization. *IEEE Trans Ind Inf* 2022;19(1):469–79.
- [5] Dutta H, Biswas S. Distributed reinforcement learning for scalable wireless medium access in IoTs and sensor networks. *Comput Netw* 2022;202:108662.
- [6] Akhloufi MA, Arola S, Bonnet A. Drones chasing drones: reinforcement learning and deep search area proposal. *Drones* 2019;3(3):58.
- [7] Dogru S, Baptista R, Marques L. Tracking drones with drones using millimeter wave radar. In: *Fourth Iberian Robotics Conference*. 2020, p. 392–402.
- [8] Hu J, Zhang H, Song L, Schober R, Poor HV. Cooperative internet of UAVs: Distributed trajectory design by multi-agent deep reinforcement learning. *IEEE Trans Commun* 2020;68(11):6807–21.
- [9] Yuna WJ, Junga S, Kima J, Kimb JH. Distributed deep reinforcement learning for autonomous aerial eVTOL mobility in drone taxi applications. *ICT Express* 2021;7(1):1–4.
- [10] Tan Z, Karaköse M. Proximal policy based deep reinforcement learning approach for swarm robots. In: *2021 Zooming Innovation in Consumer Technologies Conference*. 2021, p. 166–70.
- [11] Jacopo P, Hehui Z, Siqi Z, James X, Amanda P, Angela PS. Learning to Fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 2021, p. 7512–9.
- [12] Venturini F. Distributed deep reinforcement learning for drone swarm control. University of Padova; 2021.
- [13] Park S, Kim HT, Lee S, Joo H, Kim H. Survey on anti-drone systems: components, designs, and challenges. *IEEE Access* 2021;9:42635–59.
- [14] Tan Z, Karaköse M. Comparative evaluation for effectiveness analysis of policy based deep reinforcement learning approaches. *Int J Comput Inf Technol* 2021;10(3):1–15.
- [15] Tan Z, Karaköse M. On-policy deep reinforcement learning approach to multi agent problems. In: *Interdisciplinary Research in Technology and Management*. 2021, p. 369–76.
- [16] Heess N, TB D, Sriram S, Lemmon J, Merel J, Wayne G, et al. Emergence of locomotion behaviours in rich environments. 2017, arXiv Preprint arXiv: 1707.02286.
- [17] Unlu E, Zenou E, Riviere N, Dupouy PE. Deep learning-based strategies for the detection and tracking of drones using several cameras. *IPSP Trans Comput Vis Appl* 2019;11(1):1–13.
- [18] Wyder PM, Chen YS, Lasrado AJ, Pelles RJ, Kwiatkowski R. Autonomous drone hunter operating by deep learning and all-onboard computations in GPS-denied environments. *PLoS One* 2019;14(11):1–18.
- [19] Kayid AM. Distributed deep reinforcement learning for large scale robotic simulations. Müh: Department Of Informatics, Technische Universität München; 2019.
- [20] Liu B, Ding Z. A distributed deep reinforcement learning method for traffic light control. *Neurocomputing* 2022;490:390–9.
- [21] Nahhas A, Kharitonov A, Turowski K. Deep reinforcement learning techniques for solving hybrid flow shop scheduling problems: Proximal policy optimization (PPO) and asynchronous advantage actor-critic (A3C). In: *Proceedings of the 55th Hawaii International Conference on System Sciences*. 2022.
- [22] Espeholt L, Soyer H, Munos R, Simonyan K, Mnih V, Ward T, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In: *International Conference on Machine Learning*. 2018, p. 1407–16.
- [23] Wang M, Wu L, Li J, Wu D, Ma C. Urban traffic signal control with reinforcement learning from demonstration data. In: *2022 International Joint Conference on Neural Networks*, vol. 18, no. 23. 2022, p. 1–8.
- [24] AlMahamid F, Grolinger K. Autonomous unmanned aerial vehicle navigation using reinforcement learning: a systematic review. *Eng Appl Artif Intell* 2022;115:105321.
- [25] Mnih V, Kavukcuoglu K, Silver D, Graves A, Antonoglou I, Wierstra D, et al. Playing atari with deep reinforcement learning. 2013, arXiv:1312.5602.
- [26] Sutton R, Barto A. Reinforcement learning: An introduction. London: MIT Press; 2015.
- [27] Mnih V, Kavukcuoglu K, Silver D, Rusu A, Veness J, Bellemare M, et al. Human-level control through deep reinforcement learning. *Nature* 2015;518(7540):529–33.
- [28] Ong HY, Chavez K, Hong A. Distributed deep Q-learning. 2015, arXiv Preprint arXiv:1508.04186.
- [29] Alferaidi A, Yadav K, Alharbi Y, Razmjooy N, Viriyasitavat W, Gulati K, et al. Distributed deep CNN-LSTM model for intrusion detection method in IoT-based vehicles. *Math Probl Eng* 2022;1–8.
- [30] Xie J, Peng X, Wang H, Niu W, Zheng X. UAV autonomous tracking and landing based on deep reinforcement learning strategy. *Sensors* 2020;20(19):5630.
- [31] Zhao J, Liu H, Sun J, Wu K, Cai Z, Ma Y, Wang Y. Deep reinforcement learning-based end-to-end control for UAV dynamic target tracking. *Biomimetics* 2022;7(4):197.
- [32] Wang T, Qin R, Chen Y, Hichem S, Chang C. A reinforcement learning approach for UAV target searching and tracking. *Multimedia Tools Appl* 2019;78:4347–64.
- [33] Moon J, Papaioannou S, Laoudias C, Kolios P, Kim S. Deep reinforcement learning multi-UAV trajectory control for target tracking. *IEEE Internet Things J* 2021;8(20):15441–55.
- [34] Chen YJ, Chang DK, Zhang C. Autonomous tracking using a swarm of UAVs: A constrained multi-agent reinforcement learning approach. *IEEE Trans Veh Technol* 2020;69(11):13702–17.
- [35] Ma B, Liu Z, Dang Q, Zhao W, Wang J, Cheng Y, et al. Deep reinforcement learning of UAV tracking control under wind disturbances environments. *IEEE Trans Instrum Meas* 2023;72:1–13.
- [36] Bertoin D, Gauffriaux A, Grasset D, Gupta JS. Autonomous drone interception with deep reinforcement learning. In: *12th International Workshop on Agents in Traffic and Transportation (ATT 2022) in Conjunction with IJCAI-ECAL*. 2022, p. 3173.
- [37] Raffin A, Hill A, Gleave A, Kanervisto A, Ernestus M, Dormann N. Stable-baselines3: Reliable reinforcement learning implementations. *J Mach Learn Res* 2021;22(1):12348–55.